Efficient Graph Query Processing in PostgreSQL using Recursive SQL and Indexing

Sameer Patil	Abhi Jain	Jatsya Jariwala	Harsh Suthar
Roll No: 23B1035	Roll No: 23B0903	Roll No: 23B0951	Roll No: 23B1067

1 Project Overview

Graph-structured data is fundamental to applications such as social networks and contact tracing systems. While specialized graph databases exist, most production systems rely on relational databases like PostgreSQL. This project investigates how graph queries can be efficiently expressed and executed within PostgreSQL using standard SQL constructs.

We model graphs using relational tables and implement core graph operations such as reachability, path discovery, and neighborhood queries using recursive Common Table Expressions (`WITH RECURSIVE`). We further study the impact of indexing and query planning decisions on performance.

The focus of the project is on implementation and empirical evaluation of query execution behavior within a relational query processor.

2 Objectives

- Model graph-structured data using normalized relational schema.
- Implement graph traversal using recursive SQL (fixpoint computation).
- Analyze query execution plans using PostgreSQL's cost-based optimizer.
- Evaluate the effect of indexing and join strategies on performance.
- Demonstrate applicability through real-world graph workloads.

3 High-Level Implementation Plan

3.1 Graph Representation

We use the following schema:

- **Nodes(id PRIMARY KEY, attributes)**
- **Edges(src, dst, FOREIGN KEY references Nodes)**

Edges are stored as adjacency lists, enabling efficient join-based traversal.

3.2 Query Implementation

Graph traversal is implemented using recursive CTEs:

- Base case initializes frontier nodes.
- Recursive step performs joins over `Edges` to expand the frontier.
- Duplicate elimination ensures termination and correctness.

3.3 Indexing and Optimization

- B-tree indexes on `Edges(src)` and `Edges(dst)`.
- Composite indexes for join acceleration.
- Analysis of join algorithms (nested loop vs hash join) via `EXPLAIN ANALYZE`.

We study how indexing affects recursive join cost and intermediate result size.

3.4 Performance Evaluation

- Synthetic graph datasets (varying size and density).
- Metrics: execution time, number of tuples processed, query cost.
- Detailed plan analysis using PostgreSQL execution traces.

4 Real-World Use Cases

4.1 Social Network Analysis System

Users are modeled as nodes and friendships as edges. The system supports:

- Reachability queries using recursive joins.
- Mutual friend detection via self-joins.
- Friend recommendation using multi-hop traversal.
- Shortest path approximation using BFS-style recursion.

This models typical social network operations and enables analysis of multi-hop connectivity within a relational framework.

4.2 Disease Spread and Contact Tracing

Individuals are nodes and interactions are edges. The system supports:

- k-hop contact tracing using bounded recursion.
- Infection chain reconstruction via path expansion.
- Analysis of propagation patterns through iterative query execution.

This use case demonstrates how recursive query processing can model real-world propagation phenomena.

5 Reference

H. V. Jagadish et al., “Special Issue on Graph Data Management,” IEEE Data Engineering Bulletin, 2014.